Literature Review

E/21/327
Induka Rathnayake

Table of Content

Research Papers I gone through

[1] Machine Learning Embedded in Distribution Network Relays to Classify and Locate Faults

[2] New data-driven approach to bridging power system protection gaps with deep learning
DOI : 10.1016/j.epsr.2022.107863

[3] Deep-Learning Based Fault Events Analysis in Power Systems
DOI : 10.3390/en15155539

[4] Deep Learning-Based Method for the Robust and Efficient Fault Diagnosis in the Electric Power System

[5] Deep Learning for Short-Circuit Fault Diagnostics in Power Distribution Grids: A Comprehensive Review
DOI : 10.3390/computers15020076

Paper Number	Simulation Software	Grid / Test System	Voltage Level	Fault Types Simulated
1	MATLAB/Simulink	IEEE 123-bus feeder	4.16 kV	SLG, LL, 3LG
2	ATP-EMTP (HIF study), WinIGS (inter-turn fault study)	IEEE 34-bus (HIF), IEEE 13-bus (transfer learning), 20 MVA transformer model	24.9 kV / 4.2 kV	High-impedance fault, transformer inter-turn fault
3	MATLAB/Simulink (cyber-physical testbed)	500 kV 4-substation cyber-physical model (2 transmission lines)	500 kV	SLG, DLG, DLL, TLG (10 types per line)
4	PSCAD	3-transmission line radial system with 1 load bus	per-unit	SLG (A/B/C), DLG, LL, 3-phase
5	Multiple: (MATLAB/Simulink, PSCAD, ATP/EMTP, WinIGS, OpenDSS, OPAL-RT	IEEE 13, 14, 33, 37, 123 buses and custom microgrids	Various (LV to HV)	All short-circuit types incl. HIF

Note : All this papers are attached on next to this file on the portfolio.

1. Evolution of Power System Fault Detection

Let's find technical foundation of the research by analyzing conventional physical protection methods to modern, artificial intelligence driven approaches within power distribution networks.

1.1 Traditional Impedance and Overcurrent Relaying

Historical Baseline
Historically, distribution networks relied heavily on deterministic numerical relays like directional overcurrent and distance impedance relays.

Operational Mechanism
These conventional fault detection methods oprate on fixed mathematical thresholds, waiting for an amplitude threshold to be crossed before tripping a circuit breaker [5]. Also, they rely on heavy mathematical calculations. This result in delayed actions against power quality disturbances [4].

Limitations & Vulnerabilities
An important protection gap exists in these traditional systems. They are incapable of providing 100% protection for all system events. Specially, they fail to reliably detect High Impedance Faults (HIFs) and transformer inter-turn faults since the fault current remains too low to trigger traditional breaker thresholds [2].

1.2 Shift Towards Data-Driven Paradigms

The Methodological Shift
To overcome the limitations of traditional hardware and the slow nature of human-in-the-loop manual analysis, modern literature shows a definitive shift toward data-driven paradigms. Deep learning allows the extraction of features from massive amounts of data to delineate subtle differences in electrical waveforms under faulty conditions [4].

Automated Decision Making
Instead of relying on rigid amplitude thresholds, machine learning algorithms perform automated, self-learning monitoring and decision-making analysis [1]. This approach recognizes the actual shape of a fault waveform, complementing traditional protection technology so that faulty conditions are adequately distinguished from normal operational transients [2].

Algorithmic Comparison for Embedded Relays:

Traditional ML (e.g., SVM)
Foundational studies propose embedding SVMs inside distribution relays to classify and locate faults.

- Pros: Highly lightweight and computationally efficient for edge hardware.
- Cons: Relies on manual feature extraction and struggles with highly complex, non-linear fault signatures.

Deep Learning (e.g., CNN, CNN-LSTM)
Advanced studies propose processing raw time-series data.

- Pros: Achieves near-perfect accuracy and automatic spatiotemporal feature extraction.
- Cons: Extremely high memory footprint, creating a severe deployment bottleneck for constrained edge relays.

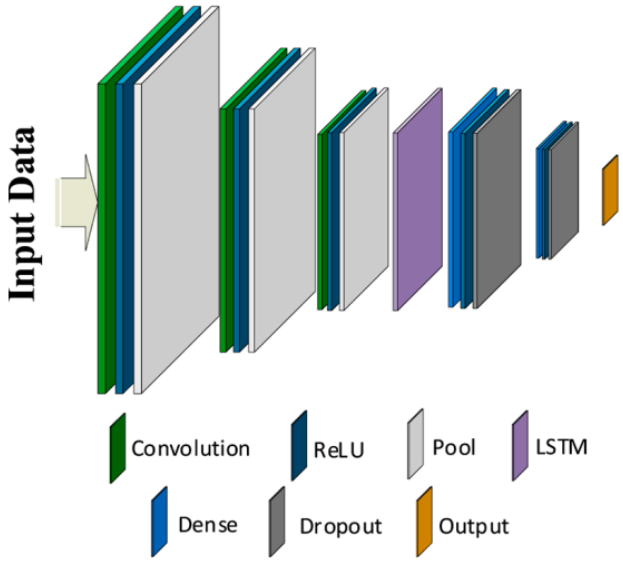


Figure 1 : A Deep Neural network [2]

1.3 Challenges in Active Distribution Networks

The DER Complication

The necessity for intelligent fault diagnostics is increasing system complexity grows with the integration of Distributed Energy Resources (DERs) [5]. Inverter-based DERs inherently limit fault currents during a short circuit, this phenomenon distorts traditional impedance-based analysis and renders classical numerical relays largely ineffective [5].

This points a massive research gap regarding real-time responsiveness and deployment scalability. Developing an embedded diagnostic model that balances the high accuracy of deep learning with the low latency required by physical edge hardware is the primary objective of this project.

2. Time-Series Data Generation and Preprocessing

Let's categorizes how existing researchers generate, format and prepare transient data for neural networks. Because deep learning models require massive datasets to learn fault signatures and methodology behind data creation is just as critical as the neural network architecture itself.

2.1 Electromagnetic Transient Simulation

Collecting physical, high-frequency fault data from active power grids is extremely dangerous and rarely permitted by utility companies. Also, real world data lacks the volume and variety required to train deep learning models. Therefore, these projects relies entirely on software simulation to build training datasets.

Simulation Engines

PSCAD / EMTDC identified as the absolute gold standard for generating high-fidelity, microsecond-level transients. It accurately models the non-linear behavior of power electronics and DERs during a fault. PSCAD seamlessly exports data via the IEEE COMTRADE standard, which perfectly mimics the data format of physical relays in the field.

MATLAB / Simulink is widely used for prototyping due to its built-in deep learning toolboxes. While excellent for extracting data directly into memory via APIs, its generalized component libraries may lack highly specialized electromagnetic transient accuracy of PSCAD.

Parameter Variation makesure neural network is robust, literature shows simulations must be run thousands of times while varying the Fault Inception Angle (FIA), fault resistance and fault location.

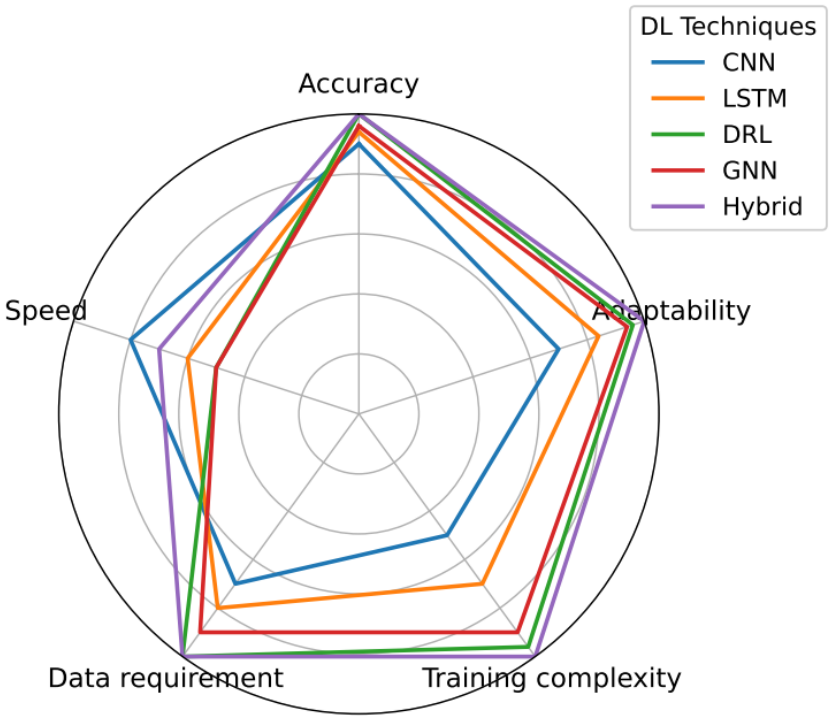


Figure 2 : Comparative performance analysis of DL techniques used in power distribution fault analytics [5]

[i.e. More complicated analysis on this had done in '*Data Export Methods Research*' and '*Data Extraction Reliability Research'* {in week 3}]

2.2 Signal Processing and Feature Engineering

Deep learning algorithms cannot continuously ingest an infinite stream of data. So, continuous time-series signal must be preprocessed and segmented.

Data Windowing

Capture specific 'windows' of data. A common approach is extracting a 1/4 or 1/2 electrical cycle (e.g., 10 milliseconds of data at 50Hz) immediately following fault trigger.

Dimensionality Transformation (1D to 2D):

Data transforms raw 1D voltage and current signals into 2D matrices using techniques like Discrete Wavelet Transform (DWT) or by stacking three-phase signals into a multi-channel array.

- Pros: Reveals hidden frequency-domain structures, boosting accuracy of CNN models in classifying complex faults.
- Cons: Signal transformation is highly computationally expensive. Converting signals to 2D matrices adds severe processing latency and directly threatening the strict 10-20ms tripping deadline required by hardware relays.

[i.e. Comprehensive research on DL models had did by my project mates.]

2.3 Data Integrity, Normalization and Noise Injection

Normalization Protocols

A distribution network's voltage may operate in thousands of volts (kV) and current operates in hundreds of amps (A). Feeding these raw numbers into a neural network causes gradient explosion and failure to converge. So, it should apply Min-Max scaling or Z-score standardization to normalize all inputs between 0 and 1 or -1 and 1.

The 'Sim-to-Real' Gap

Simulated data is mathematically perfect, which causes models to overfit. Real-world physical sensors experience electromagnetic interference and measurement jitter.

Noise Injection

To bridge this gap, advanced studies deliberately corrupt their simulated datasets by injecting Additive White Gaussian Noise (AWGN). By training the model on data with varying Signal-to-Noise Ratios (SNR, e.g., 20dB to 50dB), resulting algorithm becomes highly resilient to real-world sensor inaccuracies.

3. Evaluated Algorithmic Architectures for Fault Classificatin

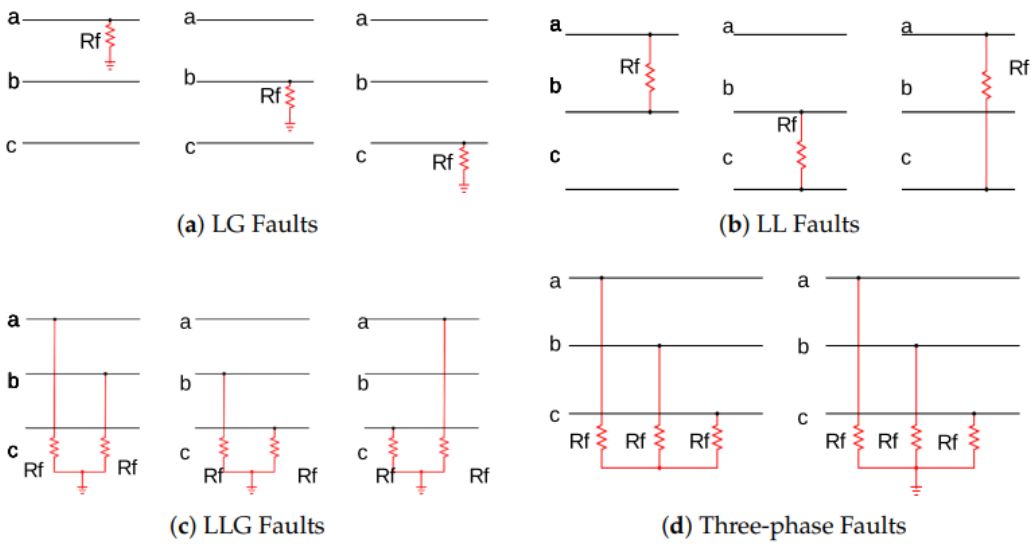


Figure 3. Different short-circuit fault types in a three-phase system [5]

3.1 Traditional Machine Learning (SVM, Random Forests, Decision Trees)

Baseline is, before widespread adoption of deep learning, automated fault detection relied heavily on classical algorithms like Support Vector Machines (SVM) and Decision Trees to classify faults [1].

- Pros: These models are extremely lightweight. They require minimal RAM and computational power, making them easily deployable on basic, resource-constrained microcontrollers embedded within distribution network relays [1].
- Cons: Classical ML algorithms cannot process raw transient waveforms directly. They require heavy manual feature engineering (e.g., Principal Component Analysis or Discrete Wavelet Transforms) before classification can occur. This adds processing delays [4, 5]. Also, comparative literature shows that SVMs generally underperform against deep learning (e.g., SVM achieving 96.8% accuracy versus CNN achieving >99% on the exact same dataset) [4].

3.2 Convolutional Neural Networks (CNN)

CNNs are widely utilized in the literature to automatically extract spatial and structural signatures directly from voltage and current matrices without the need for manual feature extraction [3, 4].

- Pros : End-to-end learning capabilities. Literature demonstrates that a 1D-CNN can achieve an accuracy of over 99% even on heavily down-sampled data (e.g., 50 Hz). Processing low-sampled data drastically reduces the inference time down to milliseconds making 1D-CNNs highly viable for real-time relay deployment [4].
- Cons : Some studies utilize 2D-CNNs that require transforming 1D time-series data into 2D arrays or colorized images before processing [3]. This signal transformation adds preprocessing latency overhead that is unsuitable for edge devices.

3.3 Long Short-Term Memory (LSTM) Networks

LSTMs are an advanced type of Recurrent Neural Network (RNN) designed specifically to handle sequential, chronological time-series data by maintaining an internal memory state.

- Pros : LSTMs are naturally suited for fault transients and easily recognizing temporal dependencies and evolution of a fault over time without explicit feature extraction [5].
- Cons : High computational latency. Because LSTMs process data sequentially step-by-step, they suffer from inherently slower training and inference speeds compared to CNNs. This sequential processing bottleneck risks missing the strict 10-20 millisecond relay tripping deadline required for hardware protection systems [5].

3.4 Hybrid Architectures (CNN-LSTM)

Recent state-of-the-art papers combine CNN layers (acting as a spatial encoder to extract waveform shape) with LSTM layers acting as a temporal decoder to track fault sequence evolution [2, 5].

- Pros : Achieves absolute highest accuracy and robustness. These models easily bridge complex protection gaps such as detecting High-Impedance Faults (HIFs) [2]. They also successfully utilize Transfer Learning to overcome real-world data scarcity by pre-training on simulation data [2].
- Cons : Extremely computationally heavy. Deploying a CNN-LSTM hybrid model on standard relay hardware is highly impractical due to its massive memory footprint, requiring specialized edge AI accelerators to function in real-time [5].

3.5 Emerging Paradigms (Graph Neural Networks and DRL)

For Network Topology analysis, Graph Neural Networks (GNN) and Deep Reinforcement Learning (DRL) are very useful in system-wide power analytics.

- Pros : GNNs are excellent for mapping entire network topologies across multi-bus systems, providing extreme robustness against network reconfigurations and variations in DER penetration [5].
- Cons : These models require multi-node data aggregation (e.g., multiple PMUs) and centralized cloud computing. Their compute requirements make them fundamentally incompatible with independent, localized low-power edge relays [5].

4. Edge Deployment and Relay-Embedded Constraints

Let’s deep dive in actual physical deployment of deep learning models onto relay hardware. It shifts the focus from theoretical algorithm accuracy to the practical feasibility of embedding AI inside edge devices.

4.1 Computational Latency and Inference Speed

Inference latency for a protective hardware relay is universally identified as to be effective. It must detect a fault, classify it and send a trip signal to a circuit breaker within 10 to 20 milliseconds (approximately one electrical cycle at 50Hz/60Hz) [1, 5].

Deep learning models that take longer than 20 milliseconds to compute are physically useless in the field, regardless of how accurate their predictions are. Complex models (like CNN-LSTMs often suffer sequential processing delays that risk violating this important safety deadline [2, 5].

Also, thses proves that properly optimized, lightweight architectures such as 1D-CNNs or traditional SVMs can achieve sub-millisecond inference times on live data streams, making them highly suitable for real-time hardware tripping [1, 4].

4.2 Hardware Limitations

Physical relays, intelligent electronic devices (IEDs) and distribution edge nodes rely on constrained microcontrollers (e.g., standard DSPs or ATmega/ARM architectures). These devices possess extreme memory limitations restricted to just a few megabytes or kilobytes of RAM [1].

Model Footprint

In stark contrast, highly accurate deep learning models often require gigabytes of memory to store their millions of learned parameters and weight matrices. This creates a massive hardware deployment bottleneck [5].

Compression Requirements

To successfully embed these models, the necessity of edge-optimization techniques are needed. This includes parameter quantization converting heavy 32-bit floating-point numbers to lightweight 8-bit integers and network pruning to drastically reduce the model's footprint so it fits on an edge device without suffering fatal accuracy loss [4, 5].

4.3 Real-Time Data Ingestion and Preprocessing Penalties

A physical relay in the field does not read static, pre-packaged CSV files. It continuously monitors a live stream of raw analog-to-digital (ADC) conversions.

If a proposed neural network requires complex, on-the-fly signal transformations(such as converting the live 1D data stream into a 2D wavelet matrix for image classification, edge device's CPU will bottleneck. This preprocessing penalty delays the fault classification entirely [4, 5].

To minimize latency, edge diagnostic models must be capable of directly ingesting standardized binary data streams (like the IEEE COMTRADE standard) with zero intermediate software or text conversion [3].

5. Synthesis and Identification of Research Gaps

This section synthesizes the findings of the comprehensive literature extraction, contrasting high-accuracy simulated architectures against the operational realities of hardware deployment to explicitly identify current research gaps [5].

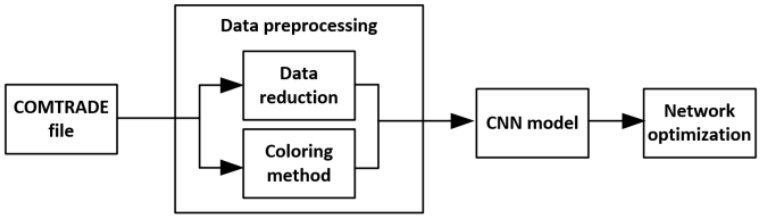


Figure 4 : Block diagram of CNN-based fault classification using COMTRADE files [3]

5. Limitations in Current Edge Implementations

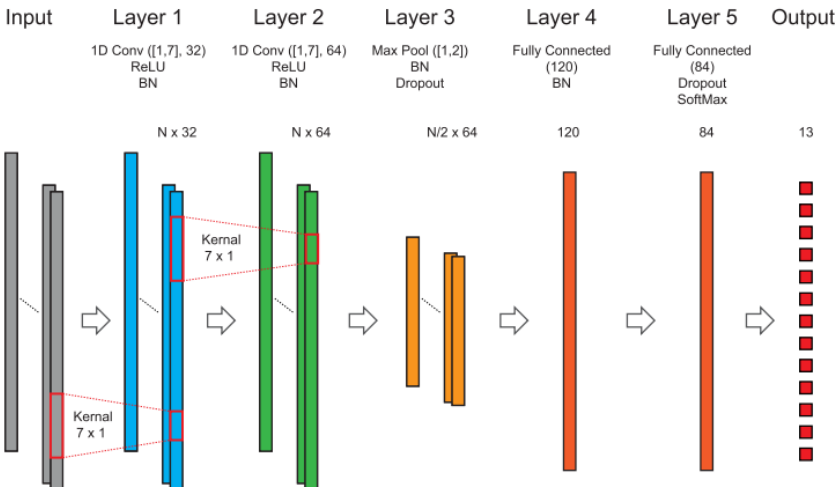
There is a major technical gap between desktop-based deep learning models and practical field protection. Advanced models such as the CNN-LSTM hybrid achieve near-perfect accuracy (98.9% - 99.3%) in software simulations, yet they demand immense computational power and memory arrays that do not exist on edge hardware [2, 5].

Preprocessing and Data Latency Overhead

Several proposed systems require heavy intermediate data manipulation before a fault can be classified. For example, converting 1D high-frequency transient arrays into 2D area-graph color images for a standard CNN requires intensive graphics/array preprocessing. This heavy preprocessing introduces computational latency, This directly violates the real-time protection constraints of electrical grids [4, 5].

Lack of Standardized Reporting

A significant majority of deep learning research focuses strictly on task-driven accuracy, completely omitting standardized reporting regarding inference latency, memory footprints and validation on physically constrained edge microcontrollers operating under live data streams.



*cation	Fault type	# of simulations	Total # of simulations	# of data for learning	Total # of data
No fault	-	1,000		12,000	
1	1	1,000		12,000	
	4	1,000		12,000	
	7	1,000		12,000	
	10	1,000		12,000	
2	1	1,000		12,000	
	4	1,000	13,000	12,000	156,000
	7	1,000		12,000	
	10	1,000		12,000	
3	1	1,000		12,000	
	4	1,000		12,000	
	7	1,000		12,000	
	10	1,000		12,000	

Figure 5 : Schematic of the CNN model for detecting fault type and location and the number of simulation data [4]